Lesson 8 Teacher's Script

Core Mechanics II: Under Pressure

VIDEO GAME CREATION

[0:00-0:05] Welcome Back

"Welcome back! Last week your enemies started moving toward you. Quick show of hands - who tuned their enemy speeds, or gave them varied movement?" [Pause, call on 1-2 volunteers]

"So far, an enemy reaching the bottom of the screen has been a non-event - it just vanishes, no consequence. Today, that changes."

[0:05-0:15] Reversing the Question: What Happens When THEY Reach YOU?

"Every lesson so far has asked one kind of question: what happens when YOUR bullet reaches an enemy? Today we ask the opposite question: what happens when an ENEMY reaches you?"

"Think back to your capstone's design document - the lose condition has always been there on paper: an enemy reaches the player's row, or the player runs out of lives. Today we actually build it."

[Discussion question] "Why do you think a game needs both a win condition AND a lose condition, instead of just one or the other?" (Draw out: stakes, tension, replayability - a game you can't lose isn't very interesting to keep playing.)

[Timing checkpoint: wrap by 0:15]

[0:15-0:35] Reusing What You Built: The Escape Path Gets Stakes

"Here's the fun part: you already wrote the code that detects an enemy reaching the bottom - you built it last week for cleanup. Today, we don't write new detection code. We just give that existing detection a real consequence."

[Demo cue: switch to Unity, open a project with working movement/escape from Lesson 7]

"Watch: here's Enemy.cs's escape branch from last week, in its Update() method - reaches the bottom, calls Destroy(gameObject), and until today, that was it. Today I add one line right before that Destroy() call: a call out to a new GameManager.RegisterEscape() method." [Add the RegisterEscape() call, show 'Game Over' appear after 3 escapes]

"Notice how small that change was. That's not an accident - it's the payoff of building things in clean, separate pieces. Last week's escape-cleanup code didn't know or care about win-

counting, so today it was easy to give it a second job that also has nothing to do with win-counting."

[Discussion question] "Can you think of a time in your own project, or in something else you've built, where old work made new work easier because it was organized well?" (Open-ended; validate any real example, even a small one.)

[Timing checkpoint: wrap by 0:35]

[0:35-0:50] Two Counters, Never Crossed: Keeping Lives and Kills Independent

"Your game now tracks two completely separate numbers: enemies_remaining (or your engine's equivalent), which only changes when you shoot something, and lives, which only changes when something gets past you. These numbers should NEVER affect each other."

[Demo cue: same Unity project] "Watch what happens if I shoot an enemy..." [shoot one] "...lives doesn't move. And if I let one escape..." [let one pass] "...enemiesRemaining doesn't move either. Two separate static methods on GameManager - RegisterKill() and RegisterEscape() - each touching only its own field, and neither one knows the other exists."

"This is worth testing explicitly in your own project today, in both directions - shoot one and confirm lives stays put, then let one escape and confirm enemies_remaining stays put. If either one moves when it shouldn't, that's this week's most common bug, and it's almost always the new code added to the wrong branch or event."

[Timing checkpoint: wrap by 0:50]

[0:50-0:55] Why We're Not Building Game Over (Yet)

"One more thing before lab: when lives hits zero today, 'Game Over' appears on screen - and then the game just... keeps running. Enemies keep spawning, you can still move and shoot. That's on purpose."

"Actually stopping the game, freezing everything, and offering a restart button is real work - enough work that it gets its own lesson, Lesson 10. For today, just like Lesson 6's win message was 'just a message' before anything fancier got added, today's lose message is the same: get the detection and the message right first, build the proper response later."

[Timing checkpoint: wrap by 0:55]

[0:55-1:00] Today's Exercise & Handouts

"For lab today, open your own project from last week - not a new one, and no new assets needed. You're adding a lives counter and a lose message, both built on top of code you already trust. If you want to try the Level 3 sound idea, the sound files are already in this week's Assets folder - no hunting for your own."

[Hand out engine-specific handouts]

"Test in both directions today - shooting shouldn't touch lives, and escaping shouldn't touch your kill counter. Your handout's Level 2 section walks through exactly how to check this."

"Questions before we start lab? ... Let's put some pressure on your player."

Lab Time Notes

- The single most important thing to verify this week: shoot an enemy (lives should NOT change) and separately let one escape (the kill counter should NOT change). Check both directions, not just that the game 'seems to work.'
- Most likely mistake: the lives decrement added to the wrong event/branch (hit-detection instead of escape-cleanup, or vice versa) - this is a copy-paste-adjacent mistake, easy to make quickly and easy to miss without deliberately testing both directions.
- No engine needs a structural bugfix this week, unlike Lesson 7 - circulate normally, no cluster needs extra attention by default.
- A student who lets every enemy through without shooting any will hit Game Over quickly - a good built-in test case, not a bug, worth pointing out if a student is confused why it happened so fast.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout (a live lives readout is a satisfying one), not given new unrelated work.